CAT-IA · Dossier del proyecto

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Este documento reúne la propuesta, informe de aplicación, plan de pruebas, manual, preparación de defensa y matriz de evaluación. La presentación y el diagrama PDF se entregan como archivos independientes.

Propuesta de proyecto: CAT-IA

Identificación y estado

Nombre: Clasificador y Asistente de Tickets con Inteligencia Artificial. Asignatura: Ingeniería de Soluciones con Inteligencia Artificial, ISY0101. Autoras: Antonella Cuvertino y Miriam Hammami. Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026. Sección: pendiente de informar. Estado: propuesta reformulada y prototipo académico; aprobación docente pendiente de acreditar.

Esta propuesta amplía el PDF original del estudiante: conserva clasificación por categorías, prioridad y salida estructurada, y añade RAG, control de contexto, validación, pruebas y documentación. La pauta permite una organización ficticia con preferencia por una real. Se adopta un escenario ficticio explícito para no atribuir a una empresa información, problemas o mediciones no verificados.

Organización

Nexo TI es una empresa ficticia chilena de soporte informático a pymes. Como supuesto de diseño, cuenta con 20 colaboradores y una mesa de ayuda de 4 operadores. Atiende problemas de acceso a portales, conectividad, consultas comerciales, facturación y sugerencias. Estas cifras describen el escenario académico; no son datos de una organización observada.

Problema y relevancia

Un operador debe leer cada ticket, distinguir categoría e impacto y consultar procedimientos repartidos en varios documentos. El uso inconsistente de políticas puede producir derivaciones incorrectas y promesas de solución que la organización no puede respaldar. Un clasificador aislado tampoco conoce los procedimientos internos vigentes ni explica de dónde obtuvo su recomendación.

Hipótesis: recuperar evidencia relevante y mostrarla junto al borrador puede reducir tiempo de triage y mejorar consistencia. No se afirma haber medido una reducción. Se propone validarla con un experimento manual asistido y sin asistencia sobre los mismos casos, alternando el orden entre participantes.

Objetivo general

Implementar y evaluar un asistente de triage que clasifique tickets y prepare respuestas basadas en fuentes internas y externas, conservando trazabilidad y revisión humana.

Objetivos medibles y criterios de aceptación

- O1: alcanzar macro-F1 de categoría de al menos 0,85 en un conjunto etiquetado y revisado por el equipo, con resultados del LLM reportados separadamente del simulador.
- O2: obtener recall documental de al menos 0,85 y precisión documental de al menos 0,70 en recuperación, documentando k, umbral, corpus y etiquetas.
- O3: verificar que todo borrador aceptado por el validador incluya citas existentes y extractos exactos; medir por separado si cada afirmación está semánticamente respaldada.
- O4: obtener al menos 0,90 de fidelidad humana y mediana de relevancia de al menos 4/5 en las respuestas del LLM. Son metas pendientes, no resultados observados.
- O5: producir abstención ante los casos sin evidencia y conservar revisión humana en todos los resultados. Registrar fallos de proveedor sin sustituirlos por simulación.
- O6: proponer una reducción de 30 % en tiempo mediano de triage frente al procedimiento manual. Validar en un piloto posterior; no incluir como logro actual.

Datos y uso

Siete documentos internos simulados describen SLA, acceso, facturación, comercial, sugerencias, conectividad y seguridad. Dos síntesis en español de documentación real de Mozilla complementan procedimientos sobre Firefox. Las fuentes están en Markdown UTF-8; el manifiesto JSON registra origen, versión, tipo, transformación y hash. Se incluyen 28 tickets sintéticos: 8 de desarrollo y 20 de prueba. No se ingieren los tickets de evaluación como conocimiento ni se usan datos personales reales.

Alcance funcional

El operador escribe un ticket, declara alcance e interrupción, y recibe categoría, prioridad, primera respuesta esperada, borrador, citas y traza. Puede variar k y filtrar fuentes para analizar el comportamiento. El sistema no envía correos, crea cuentas, reinicia equipos, devuelve dinero ni cierra incidentes. El agente tiene autonomía acotada para coordinar recuperación, consulta de política y generación; no es un agente con herramientas de ejecución arbitraria.

Restricciones

Trabajo en pareja, ejecución local, corpus pequeño, recursos de hardware limitados y plazo académico de cinco semanas. Los modelos locales requieren descarga y memoria suficientes. La autenticación empresarial, integración con una ticketera y despliegue público quedan fuera de esta versión. El prototipo minimiza datos mediante redacción básica y trazas sin ticket; no constituye una solución certificada de anonimización o cumplimiento normativo.

Justificación de LLM, RAG y agente

El LLM aporta comprensión de formulaciones variadas y redacción contextual. RAG aporta conocimiento actualizable sin reentrenar. El flujo acotado coordina tareas y permite inspeccionar qué herramienta intervino. Las políticas deterministas controlan decisiones contractuales sencillas. La combinación es pertinente porque clasificación, información empresarial y generación tienen necesidades distintas y se prueban por separado.

Plan de cinco semanas

Semana 1: seleccionar caso, levantar supuestos y proponer objetivos. Semana 2: presentar la propuesta y validar alcance; inventariar fuentes. Semana 3: implementar recuperación, prompts y contratos. Semana 4: integrar interfaz, pruebas y evaluación; revisar casos de error. Semana 5: documentar resultados, ensayar cambios en vivo y entregar. Adaptar las fechas al cronograma real: no representar estas actividades como trabajo ya realizado durante cinco semanas.

Reparto sugerido: integrante A lidera recuperación y datos; integrante B lidera API y experiencia de uso. Ambos revisan prompts, pruebas y documentación y deben poder explicar el proyecto completo. Registrar en un historial de cambios las contribuciones reales.

Referencias

Material docente aportado: propuesta-proyecto-ia.pdf, presentaciones de infraestructura y evaluación RAG y notebooks de evaluación y LangSmith. Fuentes externas de conocimiento: https://support.mozilla.org/en-US/kb/clear-cookies-and-site-data-firefox y https://support.mozilla.org/en-US/kb/firefox-cant-load-websites-other-browsers-can. API de inferencia local: https://docs.ollama.com/api/chat. Consultadas para esta implementación en septiembre de 2026.

Informe técnico de la aplicación CAT-IA

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Resumen de la solución

CAT-IA es una aplicación web local por capas que transforma un ticket en un borrador de atención verificable. FastAPI expone la API y sirve un frontend HTML/CSS/JavaScript. Python implementa ingesta, recuperación, orquestación, contratos y evaluación. Se prioriza código pequeño que el estudiante pueda explicar sobre una arquitectura de microservicios que añadiría despliegue y comunicación innecesarios para nueve documentos.

Requisitos funcionales y aceptación

- RF01. Validar tickets de 12 a 4000 caracteres, impacto y parámetros. Entradas inválidas reciben HTTP 422.

- RF02. Recuperar documentos internos y externos, con k configurable entre 1 y 8 y filtro de tipo. Ninguna fuente externa necesita red durante la consulta lexical.
- RF03. Construir contexto limitado y prompts versionados. El ticket no se concatena dentro del mensaje de sistema.
- RF04. Clasificar en Soporte Técnico, Facturación, Dudas Comerciales, Sugerencias o No determinado; validar el JSON con Pydantic.
- RF05. Determinar prioridad mediante impacto declarado. Seguridad o caída organizacional: Alta/1 hora hábil; interrupción individual o afectación colectiva: Media/4; resto: Baja/24.
- RF06. Comprobar existencia del identificador y pertenencia exacta del extracto al contexto. Una salida inválida produce abstención.
- RF07. Mostrar fuentes, puntajes, tiempos, versión del prompt y recorrido. Descargar un resultado JSON con intervención explícita del usuario.
- RF08. Registrar trazas técnicas sin texto del ticket ni borrador. Cada solicitud tiene un UUID; no existe conversación compartida entre usuarios.
- RF09. Evaluar recuperación y clasificación contra etiquetas. Dejar fidelidad y relevancia pendientes si no se realizó evaluación semántica.

Requisitos no funcionales

RNF01: ejecución reproducible en Python 3.12 y dependencias fijadas. RNF02: interfaz adaptable a móvil, etiquetas asociadas a controles y navegación por teclado. RNF03: acceso por loopback, comprobación de origen para POST y renderizado de texto con textContent para evitar interpretar HTML del modelo. RNF04: corpus y prompts identificables por SHA-256. RNF05: límite de entrada, k, contexto y salida para acotar recursos. RNF06: tiempo máximo HTTP de inferencia de 120 segundos; la latencia real se mide por equipo, no se promete un SLA técnico universal.

Arquitectura

El diagrama editable de esta etapa está en docs/arquitectura_ea1.mmd y su representación en deliverables/arquitectura_EA1.pdf. Hay dos recorridos: ingesta al arranque y consulta por ticket. En la ingesta, el manifiesto controla documentos activos, rutas y hashes; luego se fragmentan y vectorizan. En la consulta, el agente prepara el ticket, recupera evidencia, aplica política, invoca al generador, valida y presenta el borrador al operador.

Capas: presentación (web/), transporte (catia/api.py), aplicación (catia/agent.py), conocimiento (catia/retrieval.py), generación (catia/generation.py), contratos (catia/schemas.py) y configuración (config/ y prompts/). Las dependencias apuntan desde la API al agente y desde este a retriever y generador; las pruebas pueden reemplazar el proveedor sin modificar el dominio.

Datos y trazabilidad

sources.json identifica nueve fuentes. Internas: INT-SLA, INT-ACC, INT-FAC, INT-COM, INT-SUG, INT-NET e INT-SEC. Externas: EXT-COOKIE y EXT-NET. Los documentos internos son simulados. Las fuentes externas son síntesis editoriales basadas en páginas oficiales consultadas; el texto original puede cambiar y los nombres de controles dependen de la versión del navegador. El sistema no afirma tener una copia íntegra, búsqueda web en vivo ni actualización automática.

Cada fragmento conserva doc_id, id, texto, título, origen, tipo, versión, hash del documento y offsets por palabra. El hash detecta cambio de bytes, no certifica veracidad ni firma autoría. Si el contenido cambia sin actualizar el manifiesto, la ingesta falla para forzar una revisión editorial. Para aceptar cambios: revisar la fuente, actualizar versión y fecha, ejecutar scripts.source_hashes y reiniciar. No actualizar hashes automáticamente para ocultar cambios no revisados.

Chunking y recuperación

Se usan ventanas de 110 palabras con solapamiento de 20; el avance es 90. Los documentos son cortos y se evita dividir por caracteres a mitad de una palabra. El solapamiento reduce pérdida de contexto de una frontera, a costa de redundancia. Los offsets permiten reconstruir la procedencia; una mejora sería segmentar por secciones cuando el corpus crezca.

La ruta por defecto vectoriza título y contenido con TF-IDF de unigramas y bigramas, normaliza acentos y filtra una lista básica de palabras funcionales. La similitud coseno determina el ranking. TF-IDF es vectorial disperso y léxico: no debe presentarse como un modelo neuronal de embeddings. No resuelve bien sinónimos ausentes y paráfrasis lejanas. No se instala una base vectorial externa porque la matriz completa cabe en memoria; para miles de documentos se reevaluaría persistencia e índices aproximados.

La ruta hybrid solicita vectores densos reales mediante Ollama /api/embed, normaliza su norma y combina 0,55 de coseno denso con 0,45 de coseno TF-IDF. Esto es una suma ponderada, no un reranker ni RRF. El peso es inicial y requiere experimento. Los puntajes no son probabilidades de acierto. El umbral lexical 0,06 no está calibrado automáticamente para modo híbrido; debe ajustarse con desarrollo y casos fuera de dominio, pues similitudes densas positivas pueden reducir abstención.

Después del filtro se conserva hasta k=4 y hasta 6500 caracteres de texto. Se omiten fragmentos completos que no caben. Este presupuesto limita el texto documental, no cuenta exactamente todos los tokens: la envoltura JSON, metadatos y ticket también ocupan contexto. Ollama tiene num_ctx=4096 en la configuración propuesta; se debe medir tokenización y truncamiento antes de expandir el corpus.

Prompt engineering

La versión activa es v3. Después de evaluar v2 se observaron errores al distinguir cotizaciones de incidencias y phishing de otras categorías. v3 explicita categorías por intención, separa compras futuras de cargos existentes e incluye incidentes de seguridad en Soporte Técnico. El historial y los errores están en reports/OPTIMIZACION_PROMPT.md; v1 y v2 se conservan para comparación. Repetir el test observado es regresión, no evaluación independiente.

v1 es una base zero-shot con tarea, categorías y JSON. v2 añade rol, objetivo, restricciones, prioridad de fuentes, manejo de falta de evidencia, contrato, justificación breve y tres ejemplos de criterio. Los ejemplos comerciales, de facturación y ambigüedad ayudan a distinguir conceptos sin usar tickets de

prueba como ejemplos. No se pide exponer razonamiento interno; se pide una justificación breve que pueda contrastarse con citas.

build_messages guarda instrucciones en system y serializa ticket/documents como datos en user. Esto mejora separación, pero no hace al modelo inmune a prompt injection. El validador limita citas inexistentes y esquemas inválidos; una cita verdadera puede acompañar una recomendación falsa, por lo que sigue siendo necesaria revisión semántica. La prioridad interna tiene precedencia para reglas organizacionales; las fuentes externas complementan soporte técnico. El prompt establece esta jerarquía, pero no hay un detector semántico automático de conflictos implementado.

Generación y control

El esquema de evidencias se construye por solicitud: solo ofrece los ids y oraciones presentes en los fragmentos recuperados. El LLM selecciona entre esos pares mediante un JSON Schema dinámico. La validación posterior vuelve a comprobar pertenencia; limitar opciones no prueba que la oración elegida respalde la recomendación. Se redujeron metadatos enviados al LLM para evitar confundir doc_id con id de fragmento; la respuesta de la API conserva la trazabilidad completa.

OllamaGenerator invoca /api/chat con stream=false, JSON Schema, temperatura 0, num_predict=1100 y timeout de 120 segundos. Pydantic verifica campos, enumeraciones, tamaños y propiedades adicionales. Temperatura 0 reduce variación; no garantiza determinismo ni verdad. El modelo propuesto es qwen2.5:1.5b por tamaño moderado para práctica local; elección sujeta a rendimiento y revisión de licencia/model card por el equipo antes de otro uso.

DemoGenerator usa palabras clave y extractos. Es un doble funcional para ensayar interfaz y probar integraciones locales. Nunca sustituye silenciosamente un fallo de Ollama: provider_error indica fallo real, invalid_output indica salida no validada e insufficient_context indica ausencia de evidencia recuperada. El estado draft solo significa que pasó el contrato y las citas, no que un humano aprobó la respuesta.

La política de prioridad es una función determinista y se referencia a INT-SLA. Su modificación exige cambiar tanto política documental como función y pruebas: no es una regla aprendida ni inferida del texto por el LLM. La app muestra horas hábiles de primera respuesta, sin convertirlas en fecha ni considerar feriados.

API

GET / sirve la interfaz. GET /api/health muestra modo, modelo, configuración y tamaño del corpus; no comprueba por sí solo que Ollama pueda inferir. GET /api/sources devuelve el manifiesto. POST /api/tickets/analyze recibe text, impact, service_down, security_incident, top_k opcional y source_filter. /docs expone Swagger/OpenAPI y requiere internet para sus recursos visuales; /openapi.json no lo requiere.

Ejemplo de entrada: {"text":"Mi factura tiene un cobro duplicado","impact":"individual","service_down":false,"security_incident":false,"top_k":4,"source_filter":"all"}.

La respuesta incluye decision, priority, first_response_hours, sources, policy, steps, timings, usage, trace_id, model, mode, prompt_version y hashes. Las salidas degradadas siguen siendo una respuesta de negocio HTTP 200 con status explícito. Una falla en recuperación devuelve 503; entrada inválida 422; origen ajeno 403.

Seguridad y privacidad

La redacción básica reemplaza correos, RUT con formato conocido y secretos etiquetados como contraseña/token. No detecta todas las formas de datos personales, teléfonos o secretos sin etiqueta. Usar solo datos ficticios en la defensa. El backend registra categoría y metadatos técnicos, no el texto; la descarga JSON sí contiene un resumen y debe tratarse con cuidado. No se incluye autenticación ni autorización multiusuario: mantener bind en 127.0.0.1 y no exponer públicamente.

Decisiones y alternativas

ADR01: monolito modular frente a microservicios. Reduce operación y facilita seguir el flujo durante la defensa; la separación por módulos conserva mantenibilidad.

ADR02: RAG frente a fine-tuning. Las políticas cambian y necesitan procedencia; actualizar documentos es más directo que entrenar pesos y no requiere un dataset de entrenamiento grande.

ADR03: TF-IDF como baseline y embeddings opcionales. Garantiza reproducción sin descargar un modelo, pero se reconoce la menor cobertura semántica. Se compara antes de aumentar complejidad.

ADR04: Pydantic y citas exactas frente a aceptar texto libre. Permite manejar fallos de contrato y referencias inventadas; no prueba implicación semántica.

ADR05: política determinista y borradores humanos. Evita delegar compromisos contractuales simples al generador y mantiene el control del operador.

ADR06: trazas locales frente a LangSmith obligatorio. Evita credenciales y exportación del ticket; conserva ids, corpus, prompt, tiempos y uso. Se inspira en trazabilidad del curso, pero no afirma integrar LangSmith. Una futura integración requeriría decidir qué datos exportar.

Referencias técnicas

Material docente aportado de RAG, evaluación y LangSmith. Ollama: https://docs.ollama.com/api/chat, https://docs.ollama.com/api/embed y https://docs.ollama.com/capabilities/structured-outputs. Fuentes externas de dominio detalladas en data/sources.json. GitHub informa el retiro de GitHub Models: https://docs.github.com/en/github-models; por eso no se copia el endpoint histórico de la propuesta original.

Plan de pruebas y evaluación

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Propósito y niveles

Las pruebas comprueban contratos y comportamiento del software. La evaluación de recuperación comprueba los documentos obtenidos. La evaluación semántica comprueba contenido generado. Ninguna de estas capas sustituye a las otras. Las evidencias automáticas están en reports/tests.xml, reports/coverage.json y los JSON de evaluación; el resumen PDF se genera a partir de esos archivos.

Casos automatizados

- P01. Entrada vacía, corta, de espacios o extensa: rechazo Pydantic/422. RF01.
- P02. Matriz de impacto e interrupción: prioridad y horas de primera respuesta correctas. RF05.
- P03. Correos, RUT y secretos etiquetados: redacción antes de generación. RNF03.
- P04. Fragmentos de 30 palabras y solapamiento 8: límites y continuidad comprobables. RF02.
- P05. Corpus vacío, solapamiento inválido o ruta fuera de data: fallo explícito. RF02.
- P06. Consulta Firefox: recuperación de fuentes internas y externas. Filtro: solo el tipo solicitado. RF02.
- P07. Consulta de astronomía sin vocabulario pertinente: abstención y ninguna cita. RF06.
- P08. Presupuesto de contexto y k: no superar límites configurados. RNF05.
- P09. Id o extracto inventado, o ausencia de citas: rechazo y estado invalid_output. RF06.
- P10. Error de proveedor: provider_error, sin cambiar a demo. RF04.
- P11. Traza guardada: no contiene texto identificador del ticket ni correo. RF08.
- P12. Instrucción hostil del ticket queda como dato en user: comprobar separación de prompt. Es prueba estructural, no prueba de inmunidad del LLM.
- P13. Contrato Ollama chat y embeddings con mocks: request, esquema, uso y normalización. No realiza inferencia real.
- P14. Recuperación híbrida con vectores controlados: ambas señales participan. No mide embeddings descargados.
- P15. API e interfaz estática: solicitud completa, estado y origen externo. RF01/RF07.
- P16. Métricas multiclase, proveedor explícito y resultados semánticos nulos. RF09.
- P17. Conjuntos dev/test sin ids o textos duplicados. No demuestra independencia semántica ni representatividad.

Ejecutar: .venv/Scripts/python.exe -m pytest --cov=catia --cov-report=json:reports/coverage.json --junitxml=reports/tests.xml. La cobertura mide líneas ejecutadas y no calidad total ni porcentaje de la pauta. Los avisos de compatibilidad de dependencias deben conservarse en la evidencia si aparecen.

Dataset y protocolo

evaluation/dataset.json contiene 8 ejemplos dev y 20 test, todos sintéticos. Sus etiquetas provienen del diseño del implementador y deben revisarse por ambos estudiantes antes de una defensa formal. Las categorías y frases son sencillas: resultados altos pueden reflejar facilidad del corpus y no generalización. No se afirma que sea una muestra aleatoria o un test ciego independiente. Agregar tickets reales requiere autorización y anonimización apropiada.

Separar el uso: dev para elegir k, umbral y prompt; test para reportar una configuración congelada. No modificar las etiquetas o ejemplos difíciles para elevar una cifra. Si se cambia tras ver test, registrar el cambio y crear otro conjunto para validación final. Registrar corpus, prompt, modelo, configuración y fecha de cada ejecución.

Métricas definidas

Accuracy categoría: aciertos / casos. Macro-F1: media no ponderada de F1 por categoría, con zero_division=0. Matriz de confusión: filas verdaderas y columnas predichas en el orden labels del JSON. Accuracy de prioridad mide la regla con impacto declarado, no comprensión de urgencia por LLM.

Precisión de contexto documental: documentos relevantes recuperados / documentos recuperados distintos. Recall documental: relevantes recuperados / relevantes etiquetados. Se deduplican fragmentos del mismo documento; estas métricas no equivalen a precisión por chunk ni a Context Precision ponderada por ranking de RAGAS. Los casos sin documentos relevantes se excluyen del promedio de precisión/recall y se usan para abstención. MRR considera la posición del primer fragmento relevante en la lista original.

Abstention accuracy compara ausencia de evidencia esperada con status=insufficient_context. Un provider_error no cuenta como abstención correcta. Integridad de citas de borradores cuenta citas aceptadas por el validador entre drafts: es una propiedad del filtro y no una estimación de fidelidad de todo el sistema. Reportar también estados y cobertura de respuestas para evitar ocultar rechazos.

Latencia p50 y p95 se calcula por ticket dentro del proceso, después de crear el índice. Excluye arranque, descarga del modelo, renderizado y percepción del usuario. Latencia demo no se atribuye a Ollama.

Evaluación humana

Completar evaluation/revision_humana.csv después de ejecutar el LLM. Descomponer cada respuesta en afirmaciones comprobables. Faithfulness = afirmaciones apoyadas por el contexto / afirmaciones comprobables; si no hay afirmaciones, marcar no aplicable y evaluar abstención, no asignar 1 automáticamente. Relevancia: 1 no responde; 2 responde escasamente; 3 parcialmente útil; 4 responde con omisiones menores; 5 responde directamente y pide lo necesario. Revisar que cada recomendación esté apoyada por su cita, que no se inventen precios o devoluciones y que el resumen no agregue hechos.

Los dos integrantes revisan primero por separado y resuelven desacuerdos dejando notas. Evitar que el único generador se autoevalúe y tomar su juicio como verdad. Un LLM juez puede añadirse como complemento, identificando modelo, prompt y sesgos, no como sustituto de esta revisión.

Experimentos reproducibles

También se registra el experimento de optimización v2 → v3. Consultar reports/OPTIMIZACION_PROMPT.md y los JSON de cada versión. La clasificación del LLM v2 quedó bajo la meta inicial; se mejoró la taxonomía con base en sus fallos y se reporta la reevaluación sin borrar el resultado anterior.

E1: recuperar con k=2 y k=4 sobre dev, manteniendo umbral y corpus. E2: comparar v1 y v2 con el mismo LLM, datos y retriever. E3: ejecutar --baseline sin documentos y comparar afirmaciones sin soporte con el RAG. E4: probar caso fuera de dominio, instrucciones hostiles, conflicto entre guía externa y política, y ambigüedad. E5: activar embeddings y documentar evaluación separada; los resultados lexical existentes no validan ese modo.

El simulador no lee el prompt para decidir: comparar v1 y v2 en demo no demuestra optimización de prompts. El experimento sin contexto de demo tampoco mide ventaja del LLM. Solo usar inferencia real para esos argumentos.

Pruebas manuales de aceptación

MA01: abrir aplicación en escritorio y móvil; comprobar lectura, foco de teclado y ausencia de desbordamiento. MA02: caso Firefox muestra fuentes de ambos tipos. MA03: ticket comercial con URGENTE permanece Baja con impacto individual. MA04: marcar caída organizacional eleva prioridad Alta/1 hora. MA05: astronomía produce abstención. MA06: detener Ollama, analizar y observar fallo explícito. MA07: descargar JSON y cotejar ids con pantalla. MA08: revisar un borrador real contra fuente y completar ficha humana. Registrar fecha, navegador, resultado y captura; no marcar estas pruebas aprobadas sin realizarlas.

Criterio de salida

La evidencia reports/llm_smoke.json registra una inferencia real. Los archivos browser_checks.json y las capturas registran las comprobaciones de interfaz efectivamente realizadas en modo demo. El informe automático incorpora la evaluación completa del LLM cuando existe su JSON. Consultar el estado de cada ejecución; la revisión humana de contenido no se marca como completada por estos scripts.

Pruebas automáticas pasan, recuperación cumple objetivos declarados, proveedor real puede ejecutar una consulta, las métricas semánticas están revisadas, ambos integrantes explican limitaciones y el caso tiene aprobación docente. Si falta inferencia real o revisión humana, la entrega es un prototipo preparado con validación parcial, no evidencia completa del mejor logro.

Manual de instalación y uso

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Qué vas a abrir

CAT-IA es una web local para un operador de soporte. La ventana de terminal mantiene el servidor activo; el navegador muestra la aplicación. Cerrar el navegador no detiene el servidor. Ctrl+C en la terminal lo detiene.

Instalación desde cero

Instala Python 3.12, habilita Python en PATH y extrae el ZIP en una carpeta de trabajo. Ejecuta instalar.bat. Esto crea .venv, instala requirements.txt y copia .env.example solo si no existe .env. Luego ejecuta iniciar.bat y abre http://127.0.0.1:8000. No compartas .venv entre equipos: créala en cada uno. No se necesitan Node, Docker ni bases de datos para la ruta por defecto.

Una instalación nueva desde el ZIP comienza en DEMOSTRACIÓN. Es útil para aprender la interfaz y funciona con las fuentes incluidas sin inferencia. No presentes esta etiqueta como modelo real. En la carpeta original de este equipo se dejó .env configurado para Ollama local; consulta su etiqueta antes de presentar.

LLM local para la evaluación

En este equipo se preparó Ollama portable dentro de .tools. Para reiniciar tras apagar el computador, ejecutar iniciar_modelo_local.bat y luego iniciar.bat. La configuración local utiliza el puerto 11435 y el modelo qwen2.5:1.5b. Esa carpeta pesada se excluye del ZIP. En un equipo nuevo se aplica la instalación estándar siguiente y el puerto 11434.

Descarga Ollama desde su sitio oficial, instala y abre la aplicación. En una terminal ejecuta ollama pull qwen2.5:1.5b. Cuando finalice, verifica ollama list. Edita .env y cambia CATIA_PROVIDER=ollama. Mantén CATIA_RETRIEVER=lexical para una primera ejecución simple. Reinicia CAT-IA. La etiqueta de configuración debe indicar LLM; la prueba real es enviar un ticket y comprobar status=draft con uso de tokens y citas, no solo ver /api/health.

Si Ollama no está escuchando, inicia su aplicación o utiliza ollama serve en otra terminal cuando corresponda. La app usa http://127.0.0.1:11434. El modelo debe coincidir exactamente con OLLAMA_MODEL. La descarga inicial requiere internet; luego el modelo puede ejecutarse localmente. No se necesita una API key para esa ruta.

Recorrido de uso

1. Escribe qué pasó y el sistema afectado. Evita datos personales reales.
2. Selecciona si afecta a una persona, equipo u organización. Marca interrupción y seguridad únicamente cuando corresponda al escenario.
3. Deja cuatro fragmentos y fuentes internas y externas para comenzar.
4. Pulsa Analizar con evidencia y espera. En modelo local puede tardar según el equipo.
5. Lee la categoría y el resumen, y verifica la prioridad contra el impacto declarado.
6. Lee el borrador y abre cada fuente. Compara que las recomendaciones estén realmente respaldadas; un extracto exacto por sí solo no basta.
7. Consulta el recorrido para distinguir recuperación, generación, validación y abstención.
8. Descarga JSON si necesitas guardar evidencia. La app no envía la respuesta a ningún cliente.

Ejemplos para practicar

Caso 1: botón Acceso web. Debe recuperar una guía externa de cookies y procedimiento interno relacionado. Caso 2: Cobro duplicado, individual y sin interrupción. Se espera Facturación y Baja, sin devolución aprobada. Caso 3: mismo caso con una caída de toda la organización declarada; discutir por qué el operador debe comprobar que esos metadatos sean coherentes. Caso 4: Sin evidencia, astronomía; se espera abstención en recuperación lexical.

Estados

draft: salida con contrato y citas validadas, aún pendiente de revisión humana. insufficient_context: no hay evidencia sobre el umbral; solicitar antecedentes o derivar. invalid_output: el modelo no devolvió estructura/citas válidas. provider_error: no se pudo completar generación, por ejemplo por conectividad. HTTP 503: falló recuperación, posible problema de embeddings. Cada uno requiere un diagnóstico diferente.

Cambiar conocimiento y parámetros

Edita una fuente Markdown, actualiza version en data/sources.json, verifica su contenido y ejecuta .venv/Scripts/python.exe -m scripts.source_hashes. Reinicia el servidor para reconstruir el índice. Para una fuente nueva agrega entrada con id único, file, title, type, origin, version y active. Si es simulada, indícalo. Para una fuente externa guarda una síntesis propia breve y referencia su URL y fecha; no marques como consultada si no la revisaste.

Para cambiar fragmentación, umbral o versión del prompt edita config/rag.json y reinicia. Cambiar top-k en la pantalla afecta solo esa consulta. Cambiar source_filter no elimina la política interna de prioridad. Para modificar SLA, actualizar catia/agent.py, data/internal/sla.md y pruebas de prioridad conjuntamente.

Problemas frecuentes

- Python no se reconoce: instalar Python y reabrir la terminal.
- Puerto 8000 ocupado: usar .venv/Scripts/python.exe -m uvicorn catia.api:app --host 127.0.0.1 --port 8001 y abrir ese puerto.
- Hash distinto: hubo un cambio de fuente; revisar antes de regenerar hashes.
- LLM configurado pero falla: comprobar Ollama, modelo descargado, memoria y endpoint. CAT-IA no oculta ese fallo con demo.
- /docs no carga sin internet: consultar /openapi.json; Swagger usa recursos CDN.
- Respuesta poco relevante: revisar fuentes recuperadas antes de editar el prompt. Aumentar k indiscriminadamente puede agregar ruido.
- Embeddings tardan al arrancar: se indexa todo el corpus. Para volver a baseline, configurar CATIA_RETRIEVER=lexical y reiniciar.

Preparación del día de presentación

El día anterior, ejecutar pruebas y una inferencia real en el mismo computador. Dejar modelos descargados, PDF a mano y ejemplos preparados. Llevar resultados anteriores identificados como evidencia guardada, sin fingir que son inferencias en vivo. Si falla el modelo, explicar la falla y mostrar demo solo como recorrido de interfaz, reconociendo que no cumple la parte de inferencia en vivo.

Guía personal de estudio y defensa de CAT-IA

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Lo primero que debes poder decir

«CAT-IA ayuda a una mesa de soporte a clasificar tickets y preparar respuestas con evidencia. El caso es Nexo TI, una empresa ficticia. El sistema recupera políticas internas simuladas y guías externas reales de Mozilla, construye contexto para un LLM, valida su salida y muestra un borrador a un operador. El operador conserva la decisión final».

Aprende esta idea con tus palabras. No memorices resultados como si fueran universales: muestra el informe que produjo tu propia ejecución. La rúbrica también evalúa resolver un cambio sin apoyo externo; debes practicar personalmente y poder trabajar sin esta guía durante ese segmento.

Diferencia con tu propuesta original

Antes: ticket → prompt → LLM → texto JSON. Ahora: validación → recuperación interna/externa → contexto → LLM → validación de contrato/citas → borrador con traza y revisión. Además, prioridad por política explícita, manejo de ausencia de datos, pruebas y métricas separadas. El aporte de RAG se encuentra antes de generar, no consiste simplemente en guardar documentos junto al código.

Conceptos que debes dominar

LLM: modelo que genera texto a partir de patrones aprendidos. No es una base de datos de la empresa y puede producir afirmaciones plausibles falsas. Prompt: instrucciones y contexto enviados al modelo. Tokens: unidades con las que procesa texto, distintas de palabras y caracteres. Ventana de contexto: capacidad limitada de entrada y generación.

Zero-shot: dar instrucciones sin ejemplos. Few-shot: incluir pocos ejemplos de criterio. En v3 hay ejemplos para distinguir contratación de soporte, reportes de seguridad y sugerencias. La prioridad se calcula fuera del LLM. No se pide razonamiento interno paso a paso, sino una justificación verificable.

RAG: recuperar, aumentar contexto y generar. Ingesta: preparar fuentes para poder buscarlas. Chunk: fragmento de documento. Overlap: palabras compartidas entre fragmentos vecinos. TF-IDF: pondera términos por frecuencia en documento y rareza en corpus; produce vectores dispersos. Embedding

denso: representación aprendida que puede aproximar relaciones semánticas. Coseno: compara dirección de vectores. Top-k: límite de fragmentos recuperados. Umbral: mínimo puntaje admitido. Ninguno es porcentaje de confianza del LLM.

Alucinación: contenido no respaldado o incorrecto. Abstención: reconocer falta de evidencia y derivar. Ground truth: etiqueta de referencia revisable; no es verdad incuestionable por estar en un JSON. Faithfulness: respaldo de afirmaciones por el contexto. Relevancia: qué tan bien responde a la necesidad. Precisión de recuperación: proporción de recuperado que sirve. Recall: proporción de lo relevante que se recuperó.

Agente acotado: coordina un conjunto definido de pasos y herramientas sin modificar sistemas externos. CAT-IA es un workflow de asistencia, no un agente autónomo de propósito general que elige cualquier herramienta mediante function calling. Explicar esta frontera muestra dominio del diseño.

Cómo seguir el código en cinco minutos

Abre catia/api.py. La ruta POST recibe un Ticket validado y llama Agent.run. Abre catia/schemas.py: aquí están longitudes, categorías y JSON esperado. Luego catia/agent.py: redact, search, priority_for, generate, validate_evidence y respuesta con traza. Este archivo es el recorrido principal de la demo.

Abre catia/retrieval.py. load_chunks lee el manifiesto, verifica hash y corta ventanas. Retriever crea la matriz TF-IDF. search vectoriza la consulta, calcula similitud, filtra, ordena y limita contexto. Si hay embeddings, calcula también la señal densa. Abre catia/generation.py: diferencia DemoGenerator de OllamaGenerator y muestra build_messages.

Abre prompts/v3.txt y señala rol, restricciones, ejemplos y formato. Abre config/rag.json y relaciona cada número con el comportamiento. Abre data/sources.json y localiza una interna y una externa. Finalmente abre tests/test_system.py y scripts/evaluate.py: explica qué comprueba una prueba y qué mide una evaluación.

Guion sugerido de exposición de 12 minutos

0:00–1:00: problema, organización ficticia y usuario. 1:00–2:00: objetivos y alcance. 2:00–3:30: fuentes y diagrama. 3:30–4:30: prompts y control de contexto. 4:30–7:30: demo con caso Firefox y caso sin evidencia. 7:30–9:00: pruebas y métricas, identificando modo ejecutado. 9:00–10:00: decisiones y límites. 10:00–12:00: cambio pequeño y conclusión sobre próximos pasos. Ajusta al tiempo asignado por el docente; este reparto no sustituye su instrucción.

Integrante A puede cubrir problema, datos y recuperación. Integrante B puede cubrir integración, validación y pruebas. Ambos deben poder intercambiar papeles y contestar cualquier pregunta.

Demo paso a paso y explicación

Abre la aplicación y di qué modo muestra. Si dice simulación, no digas que está respondiendo un LLM. Envía: «El portal no inicia sesión en Firefox. ¿Cómo reviso cookies y datos del sitio?». Explica qué es un ticket, de dónde salen los documentos y qué diferencia hay entre política y guía externa. Muestra el puntaje y aclara que es similitud.

Lee el borrador y encuentra una cita en el fragmento. Explica que comprobar una copia exacta ayuda a detectar referencias inventadas, pero que tú verificas si esa evidencia realmente sostiene la recomendación. Muestra la traza: id, prompt y corpus. Después prueba «Distancia entre Júpiter y Neptuno». Describe por qué abstenerse es mejor que inventar una respuesta empresarial.

Prueba «URGENTE quiero conocer sus planes». Mantén impacto individual y sin caída. Prioridad esperada Baja. Explica que las categorías las propone el generador y que la prioridad proviene de una regla; esto reduce variación para una decisión simple y contractual. Las horas mostradas son de primera respuesta, no de reparación garantizada.

Cambios en vivo para practicar sin ayuda

Ejercicio 1: el docente pide reducir ruido. Cambia top-k de 4 a 2 en la pantalla, repite el mismo ticket y observa qué evidencia se pierde. Si necesitas medir, ejecuta scripts.evaluate --split dev --top-k 2 y compara con k4. No afirmes que menos k siempre mejora: puede bajar recall.

Ejercicio 2: el docente pide no inventar precios. Localiza prompts/v3.txt, refuerza la instrucción de solicitar una cotización cuando el contexto no incluya importes, guarda y repite. El prompt se lee por solicitud; cambios de config necesitan reinicio. Verifica también INT-COM: si la fuente no contiene precio, agregarlo al prompt como invento no resuelve el problema.

Ejercicio 3: llega una política nueva. Crea un Markdown breve de fuente interna simulada, añade su entrada al manifiesto con un id nuevo, versión y origen. Revisa y ejecuta scripts.source_hashes. Reinicia. Consulta una frase distintiva de ese documento y muestra id y versión. Para retirar la fuente anterior usa active=false y reinicia. No modifiques simultáneamente varios parámetros si quieres entender el efecto.

Ejercicio 4: el docente pide mayor prudencia. Cambia min_score en config/rag.json, por ejemplo de 0,06 a 0,12, reinicia y compara. Es un experimento, no un valor óptimo universal. Muestra tanto un caso conocido como uno fuera de dominio; explica el costo de abstenerse de más.

Ejercicio 5: cambia el SLA. Debes modificar priority_for, el documento INT-SLA y sus pruebas; después actualizar hash y ejecutar pytest. El sistema no interpreta una edición de SLA en texto para cambiar la función. Esta limitación es deliberada y debe decirse claramente.

Ejercicio 6: falla el proveedor. Detén Ollama, envía un ticket conocido y explica provider_error. No cambies a demo y lo presentes como recuperación transparente del LLM. Si necesitas continuar el recorrido, anuncia explícitamente el modo de práctica.

Banco de preguntas con respuestas ancladas al proyecto

1. ¿Por qué elegiste este caso? Porque triage exige clasificación, consulta documental y comunicación, tareas separables con evidencia observable. El escenario es ficticio; el impacto organizacional se plantea como hipótesis.

2. ¿Por qué RAG y no solo prompting? Un prompt general no contiene políticas ni guías concretas. RAG recupera fragmentos y permite mostrar procedencia. No garantiza verdad si la fuente está mal o la generación la interpreta mal.

3. ¿Por qué no fine-tuning? Las políticas pueden cambiar y necesito citas. Actualizar documentos es más sencillo en este corpus que entrenar un modelo. Fine-tuning podría servir para estilo o tareas repetidas,

pero no sustituye conocimiento trazable.

4. ¿Qué fuentes son reales? Las páginas de Mozilla. Los archivos externos son síntesis propias con URL, fecha y transformación. Las políticas de Nexo TI y los tickets son simulados.

5. ¿Por qué 110 palabras y overlap 20? Es un punto de partida para procedimientos breves: conserva un bloque interpretable con redundancia limitada. Se puede evaluar otra segmentación; no es una constante óptima.

6. ¿Usas embeddings? Por defecto TF-IDF, que es vectorización léxica dispersa. Existe una ruta con embeddings densos de Ollama, pero debo diferenciar qué ruta ejecuté y evalué. No llamar semántico al baseline.

7. ¿Por qué no Chroma o FAISS? Nueve documentos caben en una matriz en memoria. Un índice externo no aporta suficiente beneficio en esta escala. Si aumentaran corpus y usuarios reevaluaría almacenamiento, índice y costos.

8. ¿Qué pasa si k aumenta? Puede mejorar cobertura y agregar ruido o redundancia. El presupuesto de contexto puede impedir incluir todos los candidatos. Lo mediría con precisión y recall, no solo viendo una respuesta.

9. ¿Qué pasa si ninguna fuente sirve? En lexical se aplica el umbral y, sin hits, se devuelve insufficient_context. Un puntaje alto tampoco prueba relevancia; por eso hay evaluación y revisión humana.

10. ¿Qué garantiza el JSON Schema? Estructura, tipos, campos y categorías válidas. No garantiza que la categoría o recomendación sea correcta. El validador de citas añade procedencia, pero falta comprobar respaldo semántico.

11. ¿Temperatura cero evita alucinaciones? No. Reduce aleatoriedad de muestreo, pero no hace verdadera una respuesta. Necesito fuentes, instrucciones, validación y revisión.

12. ¿Por qué prioridad por reglas? El alcance y la interrupción se declaran en campos estructurados. Una tabla simple da comportamiento repetible y evita usar la palabra urgente como única señal. El operador debe comprobar los datos ingresados.

13. ¿Qué diferencia hay entre precisión y fidelidad? Precisión evalúa documentos recuperados; fidelidad evalúa afirmaciones generadas contra esos documentos. Una respuesta puede ser fiel a un documento irrelevante.

14. ¿Por qué puedes tener 100 % en clasificación demo? Porque es un dataset sintético sencillo y un simulador por palabras clave. Esa cifra no representa un LLM ni predice generalización. Reporto el modo y necesito pruebas más variadas e independientes.

15. ¿Cómo evalúas alucinaciones? Ejecuto un LLM real, divido las respuestas en afirmaciones y los integrantes revisan cuáles están apoyadas en el contexto. Completo la ficha humana; no uso citas existentes como sinónimo de fidelidad.

16. ¿Cómo evitas prompt injection? Separo instrucciones y datos, restrinjo formato, no doy herramientas de ejecución y valido referencias. Son capas parciales: una instrucción hostil puede afectar contenido y debe probarse con inferencia real. No afirmo inmunidad.

17. ¿Qué ocurre ante conflicto de fuentes? La política interna manda en decisiones organizacionales y el prompt lo indica. No hay un detector semántico automático de conflicto: el operador revisa. Para producción añadiría resolución explícita y evaluación adversarial.

18. ¿Qué guardas en la traza? Ids, modo, modelo, hashes, configuración, tiempos y categoría; no el texto del ticket. El resultado descargado es diferente y puede contener información redactada del caso.

19. ¿Cómo sabes que la fuente no cambió? Comparo hash al cargar. El hash solo verifica bytes, no autoridad ni verdad. Las revisiones deben actualizar origen, versión y fecha con criterio editorial.

20. ¿Qué falta para producción? Organización validada, datos representativos, autenticación, permisos, almacenamiento, límites de concurrencia, observabilidad, política de privacidad y pruebas reales. El prototipo local no debe exponerse como servicio empresarial listo.

Cómo responder cuando no sabes

Resultados del prompt: v2 logró macro-F1 0,78; v3 logró 1,00 en la regresión de 20 casos sintéticos. Explica que se corrigió la taxonomía tras observar errores y que repetir esos casos no demuestra generalización. En desarrollo, v3 tuvo un error y macro-F1 0,905. Los 20 casos de regresión incluyen 18 generaciones y 2 abstenciones sin inferencia. No digas «el modelo nunca se equivoca» ni «todas las respuestas son fieles».

Ejemplo con evidencia disponible: en desarrollo, k=2 obtuvo precisión 0,929 y recall 0,952; k=4 obtuvo precisión 0,893 y recall 1,000. Puedes mostrar reports/COMPARACION_K.md y explicar que aquí bajar k quitó ruido, pero también evidencia. No confundas esta comparación del retriever con una mejora del prompt o de la generación.

Di qué sí puedes demostrar y qué está pendiente. Por ejemplo: «No medí embeddings reales en este informe; aquí evalué TF-IDF. Para compararlos mantendría dataset y prompt, registraría modelo y cambiaría únicamente el retriever». Una respuesta limitada y precisa es defendible; inventar una cifra o una integración no lo es.

Checklist de dominio personal

Puedo dibujar el flujo sin mirar. Puedo localizar el prompt. Puedo agregar una fuente y reconstruir el índice. Puedo distinguir demo y LLM. Puedo explicar un fallo de recuperación frente a uno de generación. Puedo interpretar precision/recall y macro-F1. Puedo modificar k o una regla y ejecutar pruebas. Puedo reconocer lo que no fue medido. Si alguna respuesta es no, practicar ese punto antes de la defensa.

Matriz de trazabilidad de la pauta

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

IE1 · Diseño según requerimientos · 15 %

Evidencia: docs/01_propuesta_mejorada.md y requisitos RF/RNF del informe. Organización, problema, objetivos medibles, datos, restricciones y alcance explícitos. Pendiente externo: validación docente del caso ficticio y, si se exige, ajuste a organización real autorizada. No se afirma aprobación.

IE2 · Prompts verificables · 10 %

Evidencia: prompts/v1.txt, prompts/v2.txt, prompts/v3.txt y catia/generation.py:build_messages. Rol, tarea, contexto, formato, ejemplos, fuentes y abstención. La versión activa v3 aborda errores medidos de v2; se conservan reportes y justificación en reports/OPTIMIZACION_PROMPT.md. Demo no distingue prompts; usar resultados de inferencia para argumentar diferencias.

IE3 · Recuperación interna y externa funcional · 10 %

Evidencia: data/sources.json, archivos Markdown, catia/retrieval.py y prueba de recuperación de ambos tipos. La app opera con copias editoriales externas trazables, sin web en vivo. Para acreditar el flujo LLM completo debe funcionar Ollama en la presentación; la recuperación lexical ya es ejecutable sin él.

IE4 · Coherencia datos/respuestas · 10 %

Evidencia: validación de citas en catia/agent.py, abstención y evaluation/dataset.json. Reports contiene evaluación ejecutada. Pendiente para evidencia semántica completa: revisión humana de salidas reales en evaluation/revision_humana.csv. Citas exactas no bastan para afirmar fidelidad.

IE5 · Arquitectura integrada · 15 %

Evidencia: docs/02_informe_tecnico.md, módulos catia/ y recorrido del agente en pantalla. Ingesta, recuperación, contexto, generación, validación y revisión integrados. El diseño es un monolito modular con autonomía acotada.

IE6 · Diagrama entregable · 10 %

Evidencia EA1: docs/arquitectura_ea1.mmd, deliverables/arquitectura_EA1.pdf y diapositiva de arquitectura. Se representan datos internos/externos, índice, consulta, proveedor y salidas; las flechas distinguen ingesta de consulta.

IE7 · Decisiones fundamentadas · 10 %

Evidencia: ADR01–ADR06 del informe y parámetros de código/configuración. Alternativas, beneficios y límites: RAG frente a entrenamiento, TF-IDF frente a dense, reglas frente a LLM y trazas locales. Metas empresariales no se presentan como impactos medidos.

IE8 · Documentación y evidencia técnica · 10 %

Evidencia: propuesta, informe, manual, plan de pruebas, guía, diagramas, PDF y reports. Scripts permiten regenerar evaluación y artefactos. Autoras identificadas: Antonella Cuvertino y Miriam Hammami. Queda registrar aprobación docente y revisión humana cuando ocurran; las evaluaciones ejecutadas están en reports.

IE9 · Lenguaje y comentarios técnicos · 10 %

Evidencia: docstrings de módulos, justificación de límites y fórmulas del plan. Se distinguen simulación, inferencia, pruebas con mocks, métricas de recuperación y evaluación humana.

Defensa individual

Justificación 20 %: practicar ADR y alternativas de la guía. Dominio técnico en vivo 25 %: cambiar prompt, fuente, k o política sin ayuda externa. Cinco preguntas 35 %: preparar el banco completo y anclar cada respuesta al código. Comunicación 20 %: ensayar con cronómetro y terminología precisa. El proyecto entregado apoya la defensa; no reemplaza el dominio de cada estudiante ni garantiza nota.

Estado honesto

Implementación y documentación están orientadas al descriptor superior. La calificación depende de comprobar funcionamiento real, coherencia semántica, aprobación del caso y dominio individual. No marcar todos los indicadores al 100 % solo por tener archivos: consultar reports y los pendientes antes de entregar.

Preparación de la entrega

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Archivos principales

Entregar dist/CAT-IA_entrega.zip con código, requirements.txt, configuración de ejemplo, fuentes, prompts, pruebas, evaluación, documentación y PDF. El ZIP excluye .venv, .env, runtime, cachés, herramientas locales y modelos. Los binarios/modelos se instalan según el README. La presentación es deliverables/presentacion_CAT-IA.pdf; el informe y la guía están en la misma carpeta.

Antes de entregar

1. Datos registrados: Antonella Cuvertino y Miriam Hammami; docente Sebastián Ignacio Sánchez Morales; entrega 09/09/2026. Completar la sección en propuesta y presentación cuando se confirme.
2. Confirmar aprobación del caso ficticio o reformular con la organización autorizada.
3. Ejecutar Ollama y al menos una consulta real; luego evaluación v1/v2 y baseline según el plan.
4. Completar revisión humana de fidelidad y relevancia; registrar resultados sin atribuir cifras demo al LLM.
5. Ejecutar verificar.bat, revisar reports y regenerar PDF. scripts.build_documents no inventa resultados pendientes.
6. Ejecutar scripts.package y verificar instalación desde una carpeta nueva o equipo de la pareja.
7. Ensayar la demo y cambios en vivo sin apoyo externo. Ambos integrantes deben comprender el código.
8. Revisar que el ZIP no contenga secretos o datos personales reales.
9. Subir a AVA y enviar al correo del docente dentro del plazo indicado por él. El proyecto no realiza ese envío automáticamente.

Honestidad de la evidencia

La comprobación del paquete se guarda fuera del ZIP, en dist/package_check.json, con el hash del archivo verificado. El paquete incorpora su propio MANIFEST_ENTREGA.json. La prueba de extracción reutiliza las dependencias instaladas: verifica completitud de fuentes y arranque aislado, no representa una instalación en un sistema operativo diferente.

No afirmar que una organización ficticia es real, que el docente aprobó el caso sin constancia, que resultados sintéticos son observaciones empresariales ni que un mock es inferencia. Identificar asistencia utilizada conforme a las reglas del curso y asumir responsabilidad de entender, probar y defender el código. La defensa exige resolver personalmente un nuevo requerimiento.

Material del curso

Los PDF y notebooks aportados son referencias de estudio. No se redistribuyen íntegros en el ZIP. La entrega contiene código y documentos nuevos adaptados a la pauta, junto con fuentes propias simuladas y síntesis externas referenciadas. Los ejemplos de API Gateway/IDaaS aportados no se incorporan como requisitos adicionales: esta pauta evalúa LLM/RAG, y el prototipo local no necesita microservicios ni identidad empresarial para demostrar el flujo.

CAT-IA: cómo mostrar y explicar tu proyecto

Autoras: Antonella Cuvertino y Miriam Hammami.

Docente: Sebastián Ignacio Sánchez Morales. Fecha de entrega: 09/09/2026.

Guía práctica para abrir la aplicación, recorrer el código comentado y defender las decisiones. Complementa docs/05_guia_defensa.md, que contiene los conceptos y las preguntas de estudio. Practica las acciones tú mismo antes de la evaluación; el segmento de dominio técnico exige resolver sin apoyo

externo.

1. Prepara la demostración en este computador

1. Abre la carpeta C:\Users\anton\Desktop\evaluacion1-ia en VS Code. Usa Terminal > Nueva terminal; confirma que PowerShell esté situado en esa carpeta.

2. Abre http://127.0.0.1:8000. Si la aplicación ya responde, continúa con el paso 5. Evita abrir otro servidor en el mismo puerto.

3. Si está detenida, ejecuta iniciar_modelo_local.bat con doble clic. Este equipo tiene Ollama portable y el modelo preparado; deja la ventana abierta. Si el puerto 11435 ya está ocupado por el servicio existente, comprueba ese servicio antes de intentar iniciarlo otra vez.

4. Ejecuta iniciar.bat y deja su ventana abierta. Cuando indique que Uvicorn está disponible, abre http://127.0.0.1:8000. Para detener un servidor que tú iniciaste, usa Ctrl+C en su ventana.

5. Comprueba el aviso superior. Para demostrar inferencia real debe indicar MODO LLM y el modelo qwen2.5:1.5b. Si indica MODO DEMOSTRACIÓN, estás usando el simulador por reglas. El aviso refleja configuración: envía un ticket conocido para comprobar que la inferencia realmente responde.

6. En este equipo, .env debe seleccionar CATIA_PROVIDER=ollama, CATIA_RETRIEVER=lexical y OLLAMA_URL=http://127.0.0.1:11435. Edita únicamente si necesitas corregirlo; conserva los demás valores. Reinicia la aplicación después de cambiar .env. No muestres archivos con credenciales durante una exposición.

7. Deja abiertas tres ventanas: presentación PDF, aplicación en el navegador y VS Code. En VS Code prepara catia/agent.py, catia/retrieval.py, catia/generation.py, catia/config.py, catia/api.py, prompts/v3.txt y tests/test_system.py.

8. Haz un ensayo antes de la clase. En la evaluación guardada en CPU, la mediana fue cercana a 45 segundos y el percentil 95 a 71 segundos por caso. No son tiempos garantizados: reserva tiempo y explica la arquitectura mientras esperas. El botón se desactiva durante la solicitud; no recargues ni envíes consultas simultáneas.

2. Si vas a presentar desde otro computador

1. Extrae dist/CAT-IA_entrega.zip y abre la carpeta CAT-IA extraída. El ZIP contiene código y documentos, pero excluye .venv, .env, Ollama portable y los modelos.

2. Con Python 3.12 instalado, ejecuta instalar.bat. Este archivo crea el entorno virtual, instala requirements.txt y crea .env desde el ejemplo únicamente si aún no existe. La descarga de dependencias necesita internet.

3. Para practicar sin LLM, conserva CATIA_PROVIDER=demo y ejecuta iniciar.bat. Anuncia que se trata de una simulación del flujo.

4. Para usar un LLM real en ese equipo, sigue la instalación estándar indicada en README.md y descarga el modelo con el comando ollama pull qwen2.5:1.5b. Configura .env con CATIA_PROVIDER=ollama y OLLAMA_URL=http://127.0.0.1:11434, correspondiente al puerto estándar. Inicia Ollama y después CAT-IA.

5. No uses iniciar_modelo_local.bat en una extracción que no tenga .tools/ollama. Verifica una consulta real con anticipación; el día de la presentación no conviene depender de una descarga de modelo.

3. Apertura: qué decir en los primeros 40 segundos

«CAT-IA es un prototipo para una mesa de soporte de Nexo TI, una organización ficticia. Busca ayudar al operador a clasificar solicitudes y preparar respuestas con evidencia. Combina siete documentos internos simulados con dos síntesis de guías reales de Mozilla. Recupera fragmentos, consulta un LLM local y verifica el formato y las citas antes de mostrar un borrador. La persona conserva la decisión final».

Muestra la diapositiva del problema y luego la arquitectura. Explica que los beneficios organizacionales son objetivos por validar, no ahorros medidos en una empresa real. Relaciona el problema con requisitos concretos: clasificación, prioridad consistente, evidencia visible, abstención y trazabilidad.

4. Demostración principal: acceso web con fuentes internas y externas

1. En la aplicación, pulsa el ejemplo Acceso web. El texto describe un portal que no inicia sesión en Firefox y consulta por cookies y datos del sitio.

2. Selecciona Una persona y 4 fragmentos. Deja desmarcadas las casillas de servicio interrumpido e incidente de seguridad. En Opciones de recuperación, conserva Internas y externas.

3. Antes de enviarlo, explica: «El texto será recuperado contra el corpus. El impacto lo declara el operador y determina la prioridad por reglas; no se deduce automáticamente de la redacción».

4. Pulsa Analizar con evidencia una sola vez. Mientras responde, señala en el diagrama el trayecto: operador y API, agente, recuperación, contexto y LLM, validación y revisión humana.

5. Cuando termine, muestra categoría, prioridad y borrador. Para este ejemplo se espera Soporte Técnico. Con los campos anteriores la política determina Baja y 24 horas hábiles de primera respuesta. Si el LLM clasifica mal, identifica el error y revisa la evidencia; no lo presentes como acierto.

6. Lee una recomendación del borrador y busca su respaldo en Fuentes recuperadas. Señala una fuente interna y una externa si ambas aparecen. Explica qué aporta cada una: procedimiento organizacional y orientación técnica. La fuente externa es una síntesis local con procedencia; el sistema no navega la web en cada consulta.

7. Muestra un bloque de cita y comprueba que su oración aparece en el fragmento asociado. Di: «El identificador y el extracto se comprueban automáticamente. Yo reviso además si esa oración sostiene la recomendación». No confundas similitud con confianza o probabilidad de acierto.

8. Abre Ver recorrido del agente. Describe preparar, recuperar, consultar política, generar, validar y revisión. Este recorrido registra pasos del programa; no expone razonamiento interno del modelo.

9. Pulsa Descargar resultado JSON. Abre el archivo descargado en VS Code y localiza trace_id, prompt_version, prompt_sha256, corpus_sha256, sources, decision y timings. Esto permite investigar una ejecución concreta. Usa un ticket ficticio: el resultado descargado contiene más información que la traza persistida.

5. Demostración de límite: consulta sin evidencia

1. Pulsa Sin evidencia. Conserva el filtro Internas y externas y los valores de impacto del ejemplo anterior.

2. Pulsa Analizar con evidencia. La consulta astronómica no pertenece a la base de soporte; en el comportamiento verificado devuelve insufficient_context, sin fuentes recuperadas ni citas.

3. Explica: «El agente detecta que no recuperó evidencia y se abstiene antes de llamar al LLM. Su tarea es asistir con el corpus de soporte, no improvisar cualquier respuesta».

4. Abre catia/agent.py y localiza if not hits dentro de run. Relaciona esa condición con el resultado de pantalla. Muestra test_unknown_query_abstains en tests/test_system.py como evidencia automatizada del requisito.

6. Demostración opcional de prioridad y SLA

1. Usa Cobro duplicado con impacto Una persona y ambas casillas desmarcadas. Se espera categoría Facturación; la prioridad calculada es Baja, con 24 horas hábiles de primera respuesta.

2. Conserva el texto y selecciona Toda la organización junto con El servicio está interrumpido. Repite. La prioridad pasa a Alta y la primera respuesta a una hora hábil. Para un caso real el operador debe verificar que ese impacto sea verdadero; aquí es una variación didáctica.

3. Muestra priority_for en catia/agent.py: seguridad o caída de toda la organización producen Alta; caída o impacto de equipo/organización producen Media; el resto, Baja. La categoría la propone el generador, pero la prioridad sigue esta política determinista.

4. Señala que un SLA de primera respuesta no garantiza solución dentro de ese tiempo. Esta distinción está visible en la aplicación y evita promesas que el modelo no puede autorizar.

7. Recorrido del código: orden de lectura y explicación

Los archivos Python ahora tienen docstrings en clases y funciones, además de comentarios en pasos importantes. Los docstrings explican la responsabilidad; los comentarios explican criterios o límites. Lee el flujo siguiendo la entrada y la salida de cada componente, sin memorizar todas las líneas.

catia/api.py: empieza en create_app. El ciclo lifespan construye el agente una vez al arrancar. POST /api/tickets/analyze recibe un Ticket y llama a Agent.run. GET /api/health informa el modo configurado y GET /api/sources muestra procedencia. La aplicación sirve también HTML, CSS y JavaScript. Di: «La API conecta la interfaz con el caso de uso».

catia/schemas.py: muestra Ticket y Decision. Pydantic valida tipos, longitudes y categorías. Un ticket inválido recibe HTTP 422 antes del flujo principal. Evidence relaciona un fragmento con su cita. Di: «Un JSON válido sintácticamente todavía puede ser incorrecto para nuestro contrato».

catia/config.py y config/rag.json: config.py define límites y valida; rag.json contiene los valores activos. .env selecciona proveedor, modelo y recuperación. Si cambias rag.json o .env, reinicia la aplicación. En JSON no se insertaron comentarios porque el formato no los admite; sus campos están explicados en config.py y en la sección siguiente.

catia/agent.py: recorre run desde arriba: redacta patrones sensibles, recupera, aplica prioridad, busca la política obligatoria, genera, valida citas y devuelve resultado. Si falta evidencia o falla el proveedor, se abstiene con estado explícito. La traza persistida excluye ticket y respuesta. Es un flujo acotado; no administra por sí solo sistemas de clientes.

catia/retrieval.py: load_chunks lee el manifiesto, valida rutas y hashes y construye ventanas solapadas. Retriever crea el índice TF-IDF en memoria. search transforma la consulta, calcula similitud, ordena y filtra por tipo, umbral, cantidad y presupuesto. La ruta híbrida añade embeddings, pero las métricas entregadas se obtuvieron con lexical.

catia/generation.py: build_messages separa instrucciones de sistema y datos. OllamaGenerator.generate prepara un esquema con citas permitidas, llama a /api/chat y valida Decision. DemoGenerator usa reglas y extractos, sin ejecutar el prompt en un LLM. Señala ambas clases para demostrar que sabes qué motor estás usando.

prompts/v3.txt: identifica rol, taxonomía por intención, ejemplos few-shot, límites, citas y tratamiento de datos no confiables. Explica el ejemplo de cotizar soporte: la intención es comercial aunque el producto sea técnico. No agregues comentarios de programación al prompt: cualquier texto allí forma parte de la instrucción enviada al LLM.

data/sources.json y archivos de data: muestra una entrada interna y una externa, su ID, archivo, origen, versión y hash. Relaciona doc_id con el documento y source_id con el fragmento que se cita. Un hash detecta modificación de bytes; no acredita por sí solo verdad o calidad de la fuente.

web/index.html: estructura el formulario, panel de resultado y evidencia. Los IDs permiten que JavaScript encuentre cada elemento. **web/style.css:** define colores, distribución, foco visible y adaptación móvil. **web/app.js:** gestiona ejemplos, envío JSON, estados de espera, renderizado y descarga. Utiliza textContent para mostrar respuestas como texto, sin interpretarlas como HTML.

tests/test_system.py: contiene fixtures, pruebas del corpus, prioridad, abstención, contratos y API. Los mocks reemplazan llamadas externas para aislar comportamiento. **tests/test_evaluation.py:** comprueba la separación de datos y que las métricas humanas permanezcan pendientes hasta medirse. Ninguno de esos mocks demuestra calidad del LLM.

scripts/evaluate.py: recorre casos etiquetados, guarda resultados por caso y calcula métricas. **scripts/build_documents.py:** convierte documentación y evidencia existente en PDF. **scripts/package.py y check_package.py:** empaquetan archivos permitidos y comprueban integridad y arranque desde extracción. La comprobación reutiliza dependencias instaladas; no equivale a probar un sistema operativo nuevo.

Otros scripts: smoke_llm ejecuta una inferencia real; debug_generation diagnostica un caso fijo con v2; source_hashes actualiza el manifiesto después de revisar fuentes; check_browser comprueba la interfaz con un navegador preparado; check_documents audita límites de página; prepare_local_ollama es una utilidad de preparación, no parte del flujo por ticket. Los archivos __init__.py identifican paquetes Python. Los lanzadores .bat tienen comentarios REM sobre instalación, inicio y verificación.

8. Parámetros que debes saber explicar

- chunk_words = 110: cantidad máxima de palabras por fragmento. Palabras no son tokens.

- overlap_words = 20: palabras repetidas entre ventanas; el avance es 110 - 20 = 90.
- top_k = 4: máximo solicitado de fragmentos, no cantidad garantizada. La pantalla permite cambiarlo por consulta.
- min_score = 0.06: similitud mínima admitida; puede excluir evidencia débil, pero no detecta toda irrelevancia.
- max_context_chars = 6500: presupuesto del texto de fragmentos. No cuenta el prompt completo ni controla exactamente los tokens del modelo.
- semantic_weight = 0.55: peso de embeddings en modo híbrido; en lexical la señal densa no participa.
- prompt_version = v3: archivo de instrucciones seleccionado. Las versiones conservadas permiten comparar decisiones de prompting.

9. Cómo mostrar las pruebas paso a paso

1. En VS Code abre tests/test_system.py y busca test_priority_policy. Explica sus entradas parametrizadas y el resultado esperado. Busca después test_citation_rejection: altera una cita y espera su rechazo. Estos ejemplos verifican requisitos, no solo que una función devuelva algo.

2. En una terminal ubicada en la raíz del proyecto ejecuta:

.\.venv\Scripts\python.exe -m pytest -q

3. Espera el resumen. La suite verificada contiene 33 casos; muestra el resultado que obtenga tu ejecución. Si aparece un fallo, lee el caso y su aserción; no lo ocultes. Un aviso de deprecación no es lo mismo que un test fallido.

4. Para actualizar la evidencia de cobertura y JUnit, ejecuta antes de regenerar los PDF:

.\.venv\Scripts\python.exe -m pytest --cov=catia --cov-report=json:reports/coverage.json --junitxml=reports/tests.xml

5. Abre reports/RESUMEN_RESULTADOS.md y reports/OPTIMIZACION_PROMPT.md. Distingue pruebas de software, evaluación de recuperación e inferencia real. En los 20 casos sintéticos reutilizados, v3 registró macro-F1 1,00 frente a 0,78 de v2. Como v3 se ajustó después de inspeccionar esos casos, esto es regresión sobre casos conocidos, no generalización independiente.

6. Explica que precisión documental mide qué parte de lo recuperado es relevante; recall mide cuánto de lo relevante se recuperó; macro-F1 promedia el desempeño por categoría. La cobertura de líneas indica código ejecutado durante tests, no porcentaje de requisitos cumplidos o de respuestas correctas.

7. La revisión humana de fidelidad y relevancia sigue pendiente en evaluation/revision_humana.csv. Revisa afirmaciones contra contexto y completa resultados reales antes de atribuir puntuaciones humanas. La inspección cualitativa asistida disponible no sustituye esa revisión.

8. No ejecutes toda la evaluación Ollama de 20 casos durante una presentación breve. Muestra el informe guardado con su modelo, fecha y configuración, y una consulta real en vivo. Para repetir el experimento fuera de la exposición: .\.venv\Scripts\python.exe -m scripts.evaluate --provider ollama --prompt v3.

10. Cambio en vivo que puedes practicar

Requerimiento de ejemplo: reducir la cantidad de contexto. Usa Acceso web con k=4 y guarda el resultado JSON. Cambia Fragmentos a recuperar a 2, conserva texto y filtros y vuelve a analizar. Compara los IDs y documentos recuperados. Explica que menos contexto puede reducir ruido, pero perder información pertinente. No prometas que siempre mejorará la respuesta ni que llegarán exactamente k fragmentos.

Si te piden mostrar la implementación, abre Ticket.top_k en schemas.py, Agent.run donde pasa ticket.top_k al recuperador y Retriever.search donde limita hits. Para cambiar el valor global, edita top_k en config/rag.json y reinicia; recuerda que el formulario manda su propio valor y lo puede sobreescribir para esa consulta.

Requerimiento de ejemplo: cambiar el estilo del borrador. En modo LLM abre prompts/v3.txt y agrega una instrucción concreta compatible con el esquema, por ejemplo pedir tres pasos breves en suggested_reply. Guarda, repite el mismo ticket y comprueba contenido y citas. El archivo de prompt se lee en cada generación; este cambio no requiere reconstruir el índice. En modo demo no cambia la respuesta por prompting, porque ese proveedor no interpreta el prompt.

Conserva una copia del prompt antes del ejercicio y restaúrala al terminar el ensayo. Si decides entregar una versión nueva, registra el cambio y vuelve a evaluar: los informes anteriores corresponden al hash anterior del prompt. Los comentarios del código no alteran ese archivo.

11. Si algo falla durante la demostración

No abre la página: revisa la ventana de iniciar.bat y la dirección 127.0.0.1:8000. Error de puerto ocupado puede indicar una instancia ya iniciada; prueba esa instancia antes de abrir otra.

La página abre, pero aparece provider_error: la API está disponible, pero no logró una respuesta válida del proveedor. Revisa Ollama, modelo y coincidencia de puertos con .env. GET /api/health no detecta por sí solo ese fallo. Si necesitas continuar con demo, anuncia el cambio y reinicia con CATIA_PROVIDER=demo; no lo atribuyas al LLM.

Aparece invalid_output: el contenido no pasó validación. Muestra la abstención y revisa el contrato; no inventes una respuesta para ocultar el error. **Aparece insufficient_context:** examina consulta, filtro y umbral antes de asumir un fallo del generador.

Cambiaste configuración y no cambia el resultado: reinicia la API y comprueba la configuración en /api/health. El filtro y k del formulario se envían en cada ticket y pueden diferir del valor global.

Error de hash al iniciar: hubo cambios en una fuente. Revisa si eran intencionales antes de ejecutar scripts.source_hashes y reiniciar. Actualizar el hash sin revisar elimina la señal del cambio, no demuestra que el contenido sea correcto.

12. Orden sugerido para una defensa de 12 minutos

- 0:00 a 1:00: problema, organización, usuario y alcance.
- 1:00 a 2:30: fuentes, requisitos y diagrama de arquitectura.

- 2:30 a 3:30: prompt, contratos y decisiones de recuperación.
- 3:30 a 6:30: Acceso web, evidencia, recorrido y caso sin evidencia.
- 6:30 a 8:00: código de una validación y ejecución breve de pytest.
- 8:00 a 9:30: resultados, límites y revisión semántica pendiente.
- 9:30 a 11:30: cambio pequeño de k y explicación del efecto observado.
- 11:30 a 12:00: objetivos pendientes y aplicación del diseño al problema.

Este reparto es una sugerencia, subordinada al tiempo que asigne el docente. La demostración opcional de prioridad y la lectura extensa del código son material de ensayo o de respuesta a preguntas. Ambos integrantes deben poder recorrer todo el flujo, aunque dividan la exposición.

Resultados ejecutados de CAT-IA

Evidencia de software

Pruebas registradas: 33. Fallos y errores: 0. Cobertura de líneas: 94.5 %.

JUnit: reports/tests.xml. Cobertura: reports/coverage.json. Estos resultados validan software y contratos; los mocks no ejecutan un LLM.

Interfaz en navegador

Microsoft Edge headless / CDP. Modo: demo. Comprobaciones completadas: Firefox: borrador y ambas fuentes; Facturación: categoría y resultado renderizados; Sin evidencia: abstención visible; Viewport 390 px: sin desbordamiento horizontal. Capturas incluidas en reports/.

Evaluación del simulador y recuperación lexical

Fecha: 2026-09-08T03:46:46.850085+00:00. Casos test: 20. Proveedor: demo. Modelo: simulador-extractivo-v1 (sin LLM).

Accuracy categoría: 1.000. Macro-F1: 1.000. Accuracy prioridad: 1.000.

Precisión documental: 0.801. Recall documental: 1.000. MRR: 0.972.

Accuracy abstención: 1.000. Latencia interna p50: 1.05 ms; p95: 2.48 ms.

No se atribuyen estas cifras a un LLM. Dataset pequeño y sintético creado junto a la implementación; requiere etiquetas independientes y casos más difíciles.

Evaluación semántica

Fidelidad y relevancia humanas: pendientes de completar en evaluation/revision_humana.csv. El código verifica identificadores y extractos exactos, no que una recomendación esté lógicamente respaldada.

Experimento de recuperación en desarrollo

k=2: precisión 0.929, recall 0.952. k=4: precisión 0.893, recall 1.000.

En los ocho casos dev, k=2 reduce ruido y pierde parte de la cobertura; k=4 conserva más evidencia. Esta comparación respalda priorizar recall en la configuración por defecto, sin declarar que k=4 sea óptimo para otros corpus.

Evaluación del LLM real

Modelo: qwen2.5:1.5b. Prompt: v3. Casos: 20. Fecha: 2026-09-08T18:02:03.593693+00:00.

Accuracy categoría: 1.000; macro-F1: 1.000. Estados: {'draft': 18, 'insufficient_context': 2}.

Latencia interna p50: 45.42 s; p95: 71.36 s. Prueba en CPU; no extrapolar a otros equipos.

La clasificación incluye abstenciones y fallos como No determinado. La integridad de las citas se impone por contrato; la revisión semántica por los estudiantes sigue pendiente.

Comparación de prompts

Macro-F1 v2: 0.780; v3: 1.000. Se ajustó la taxonomía por intención del ticket.

El test se inspeccionó para diseñar v3: esta reevaluación es una regresión sobre casos conocidos, no prueba independiente de generalización. Ver reports/OPTIMIZACION_PROMPT.md.

Prueba real de inferencia

Modelo: qwen2.5:1.5b. Estado: draft. Duración: 45240.6 ms. Evidencia completa en reports/llm_smoke.json.

Una consulta real es una prueba de integración; no sustituye evaluación completa del conjunto test ni revisión humana.

Análisis cualitativo de una inferencia real

La prueba de integración reports/llm_smoke.json utiliza Qwen 2.5 1.5B en Ollama local. La revisión descrita aquí fue realizada con asistencia de IA durante la implementación; no sustituye la revisión humana independiente solicitada al equipo.

Hallazgos observados

El modelo identificó Soporte Técnico y resumió el problema de Firefox sin agregar una identidad. Seleccionó tres citas existentes, de EXT-COOKIE:0, EXT-NET:0 e INT-ACC:0. El esquema dinámico y el validador comprobaron que cada extracto pertenece a su fragmento.

La respuesta es general: sugiere consultar la guía, revisar conexión y extensiones y contactar soporte. La frase sobre un sitio que no se encuentre en la guía no está justificada como condición de escalamiento: las guías no son un catálogo de sitios del cliente. Esto ilustra por qué la validación mecánica de citas no equivale a fidelidad. Un operador debería eliminar esa condición y pedir el mensaje de error o confirmar el alcance.

Fallo detectado y corrección

La primera ejecución fue rechazada por el validador; se conserva reports/llm_smoke_initial_rejected.json. La instrumentación inicial conservaba el estado de fallo, no el detalle completo de la respuesta descartada, por lo que no se puede reconstruir toda la causa semántica de ese intento. Durante el ajuste se eliminó la ambigüedad entre doc_id e id de fragmento en el prompt y se construyó un esquema dinámico con ids y extractos reales. Se exigió al menos una cita cuando existe contexto. La prueba posterior pasó el contrato sin eliminar la revisión humana.

Próximo paso de evaluación

En la regresión v3 todas las categorías fueron correctas. Aun así, T13 añadió orientación sobre una cuenta bloqueada que el ticket no mencionaba. Esa recomendación existe en un documento recuperado, pero es poco pertinente para el reporte de un correo que solicita contraseña. Esto permite distinguir dos problemas: una afirmación sin respaldo y una orientación respaldada por un documento pero ajena a la necesidad concreta. El operador debería conservar el escalamiento de seguridad y retirar la orientación irrelevante.

Revisar todas las respuestas reales del dataset, no solo el ejemplo que funciona. Registrar recomendaciones no respaldadas, ambigüedades y omisiones. Completar la ficha humana con ambos integrantes. Si se modifica el prompt tras revisar estos casos, registrar la nueva versión y evaluar con casos nuevos; no llamar independiente a una evaluación utilizada para ajustar el sistema.